



EAST WEST UNIVERSITY

Department of Computer Science and Engineering

CSE475 — MACHINE LEARNING

Task 02: CNN Baselines + Attention Model Report

Group: 08

Project: Image-Based Malware Classification using CNN and Attention

Submitted by

Mst. Sadia Afrin (2022-2-60-037)

Sowhardo Monjurul Kabir (2022-1-60-317)

Boshra Benta Mahbub (2022-3-60-278)

Mehrin Mahabub Kotha (2022-3-60-284)

Submitted to

Dr. Raihan UI Islam

Associate Professor

Department of Computer Science and Engineering

East West University

Submission Date: 20/08/2026

Table of Contents

1. Introduction
2. Task 2 Objectives
3. Dataset and Experimental
Setup 4. Preprocessing Pipeline
5. Baseline Models
 - 5.1 Generalized CNN
 - 5.2 MobileNetV2
 - 5.3 EfficientNet-B0
6. Baseline Evaluation
7. Proposed CNN + CBAM Attention
 - Model 7.1 Why CBAM
 - 7.2 Architecture
8. First Results and Comparison
9. Discussion
10. Conclusion

1. Introduction

This report presents Task 2 of the CSE475 Machine Learning course project. The objective of this task is to establish representative CNN baselines for image-based malware classification and then develop a proposed CNN model enhanced with an attention mechanism. The report focuses on preprocessing, model design, baseline evaluation, the proposed attention architecture, and the first experimental comparison.

The project uses the MaleVis malware image dataset. In image-based malware analysis, malware binaries are represented as images whose visual patterns can be learned by computer-vision models. The Task 1 report describes the dataset as RGB malware images and highlights class-dependent texture, colour, banding, and structural patterns.

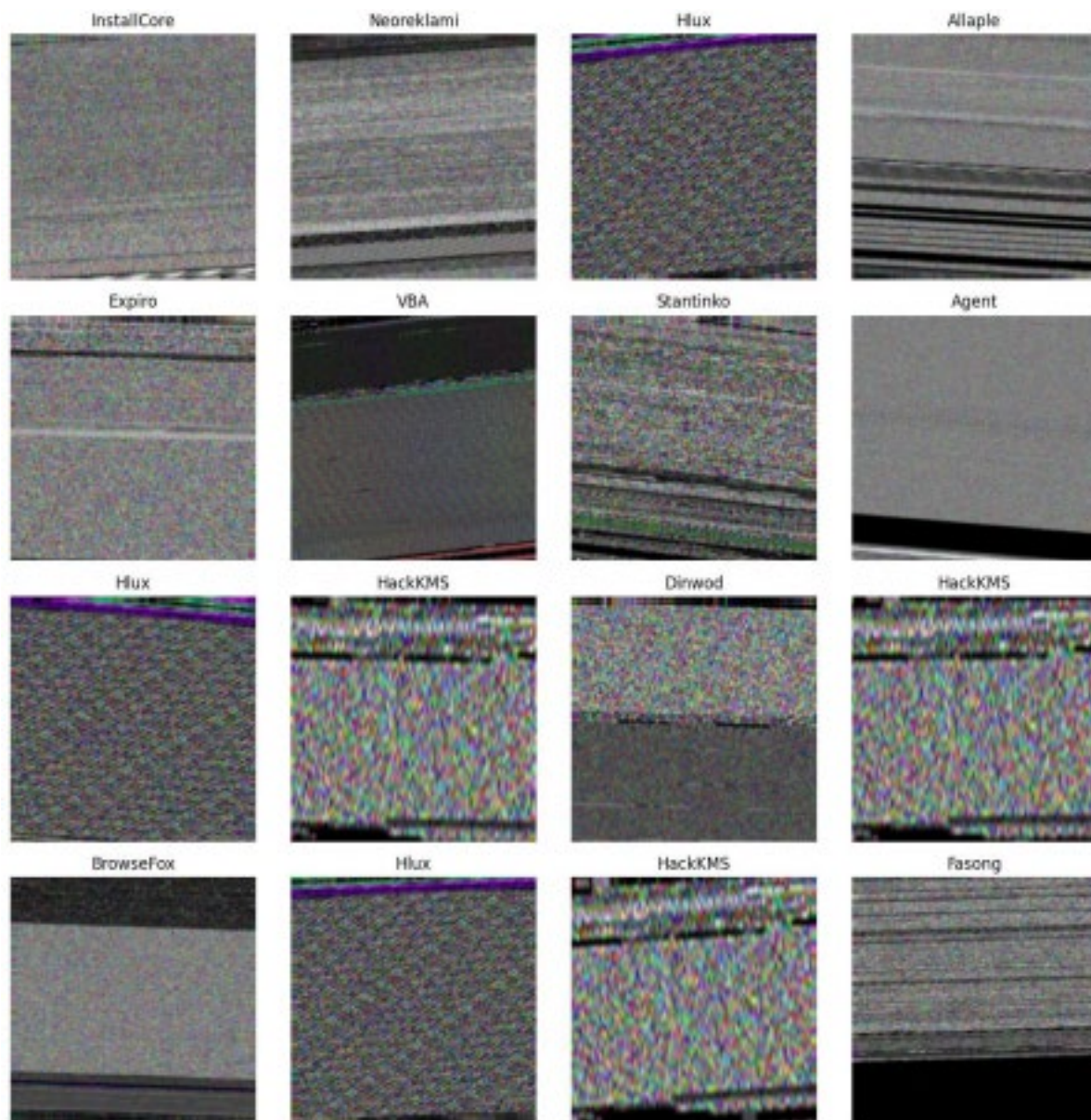


Figure 1. Representative Maleis images

Task 2 is designed around three elements: baseline CNN models, a proposed attention-based model, and a first-results comparison. The course brief explicitly requires preprocessing, full baseline evaluation, an architecture diagram for the proposed model, an explanation of the selected attention type, and an answer to whether attention helps.

2. Task 2 Objectives

- Establish representative CNN baselines for the malware-image classification problem.
- Apply a consistent preprocessing and training pipeline to make model comparisons fair.
- Evaluate the baselines using accuracy, precision, recall, F1-score, macro/weighted averages, confusion matrix, ROC-AUC and precision–recall analysis.
- Design a custom CNN with an attention mechanism and clearly identify where the attention module is inserted.

- Explain the selected attention mechanism and its expected benefit in malware image classification.
- Compare the first proposed-model result against the baseline models and determine whether attention provides an improvement.

3. Dataset and Experimental Setup

The dataset used in this project is MaleVis. The Task 1 report describes MaleVis as a multi-class RGB malware image dataset and reports 14,226 images in 26 classes, with 9,100 training images and 5,126 testing images with each image having a uniform resolution of 300×300 pixels.

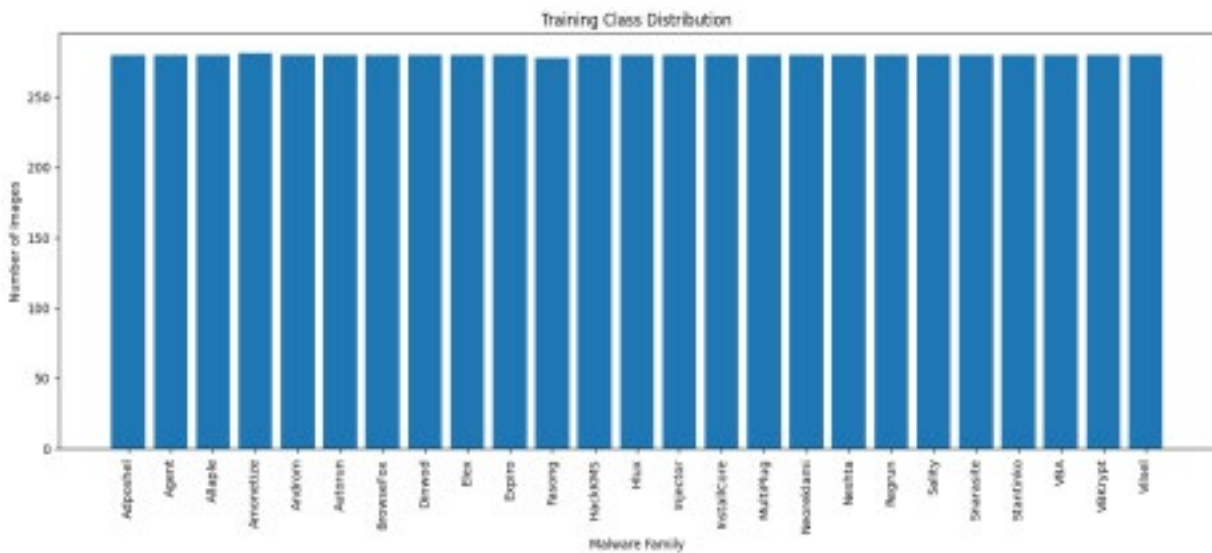


Figure 2. Dataset organization and class structure.

4. Preprocessing Pipeline

Before model training, the images are resized to the model input size, normalized, encoded into class labels, and augmented for training. The same core preprocessing strategy should be used across baselines and the proposed model to maintain a fair comparison.

- Resize images to the selected input size: 224×224 pixels.
- Normalize pixel values by dividing RGB pixel values by 255.0, producing values in the $[0, 1]$ range.
- Encode the 25 malware families using integer label IDs (0–24) with sparse categorical cross entropy.
- Training augmentation: RandomRotation factor 0.02, RandomZoom height/width factor 0.03, and RandomTranslation height/width factor 0.02.

- Validation/test transformation: resize to 224×224 , decode as RGB, cast to float32, divide by 255.0, batch at 32, and apply no augmentation.
- Data split: 6,999 training images, 1,751 internal-validation images, and 3,477 final-test images. Hash overlap checks reported zero overlap between every pair of splits.

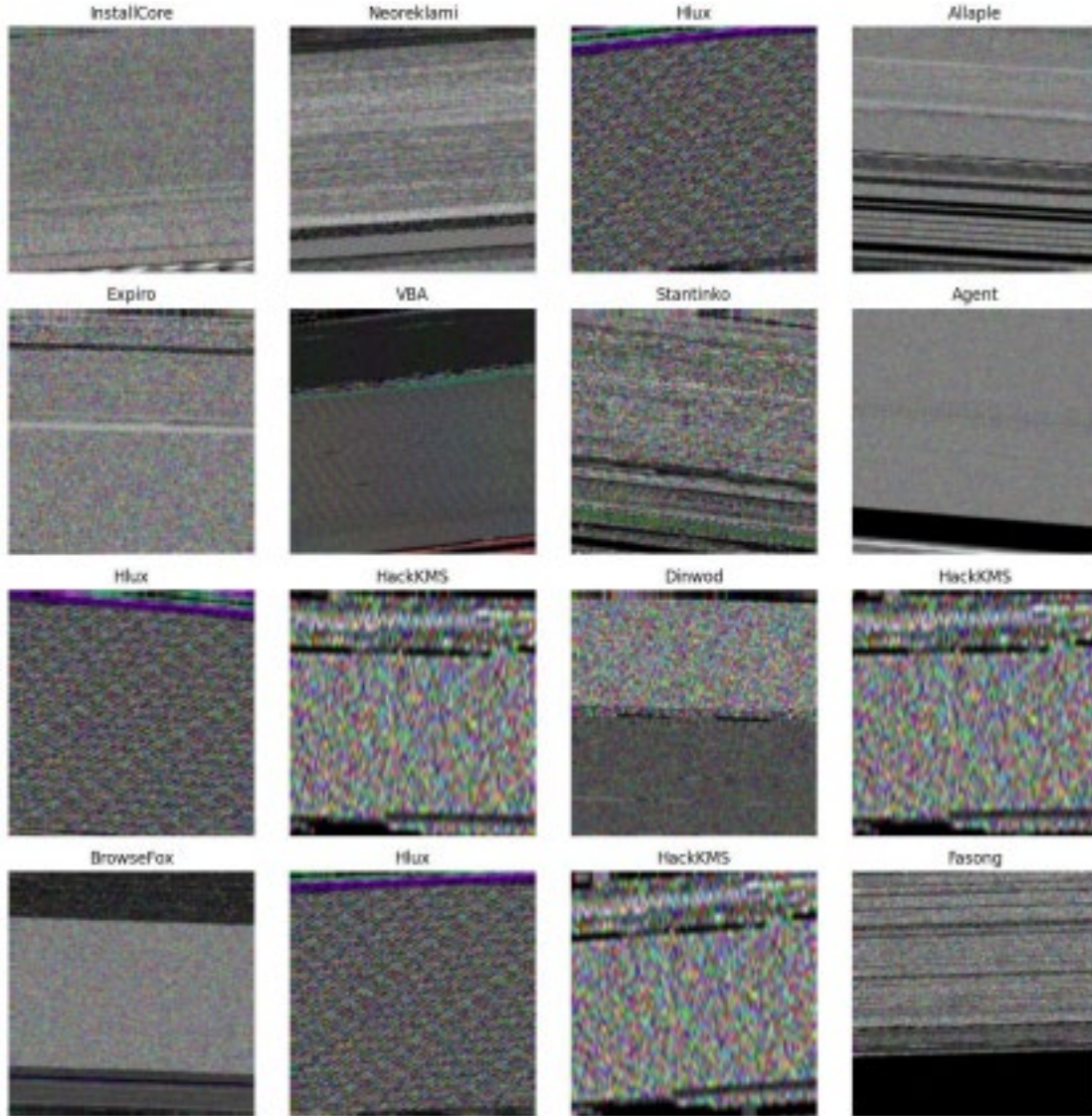


Figure 3. Preprocessing and data augmentation examples.

5. Baseline Models

The baseline stage establishes reference performance before evaluating the proposed attention model. The selected baselines in this project are a generalized/custom CNN, ResNet50, and EfficientNet-B0.

Model	Input	Classes	Optimizer/LR	Training	Parameters
Generalized CNN (Baseline)	300x300x3	26	Adam/1e-3	≤ 20 epochs; Early stopping	$\sim 420,000$
ResNet18	300x300x3	26	Adam/1e-3 $\rightarrow$ 1e-5	10 frozen+10	$\sim 11,180,000$

				fine-tune	
EfficientNet-BD	300x300x3	26	Adam/1e-3->1e-5	10 frozen+10 fine-tune	~4,010,000
Custom CNN+Attention	300x300x3	26	Adam/5e-4	<=20 epochs; Early stopping	~435,000

Figure 4. Baseline model configuration summary.

5.1 Generalized CNN

The generalized CNN is a compact convolutional baseline designed specifically for the malware image classification task. The model progressively extracts low-level and high-level visual features through convolution, normalization, activation and pooling layers, followed by global feature aggregation and a classifier.

Architecture Summary Table:

Layer Type	Output Shape	Kernel/Activation	Hyperparameters	Output Features
Input Layer	300 x 300 x 3	-	-	Raw RGB image
Conv2D_1	298 x 298 x 32	3 x 3, ReLU	Stride:1, Padding Valid	Low-level feature extraction
MaxPool2D_1	149x 149x 32	2 x 2	Stride:2	Downsampling
Conv2D_2	147x 147x 64	3 x 3, ReLU	Stride:1, Padding Valid	Mid-level feature extraction
MaxPool2D_2	73 x 73 x 64	2 x 2	Stride:2	Downsampling
Conv2D_3	71 x 71 x 128	3 x 3, ReLU	Stride:1, Padding Valid	High-level pattern representation
MaxPool2D_3	35 x 35 x 128	2 x 2	Stride:2	Downsampling
GlobalAvgPool	128	-	Spatial Averaging	Vector representation
Dense(Hidden)	128	ReLU	Dropout Rate:0.5	Fully connected representation
Output Layer	26	Softmax	Class probabilities	Multi-class prediction

5.2 MobileNetV2

A pre-trained **MobileNetV2** model was evaluated as a transfer-learning baseline for the 25-class

MaleVis malware classification task. The model used **6,999 training images, 1,751 internal-validation images, and 3,477 held-out test images**, with all input images resized to $224 \times 224 \times 3$ and a batch size of **32**. The ImageNet-pretrained backbone was initially frozen and trained with a new classification head consisting of global average pooling, dropout, and a 25-class softmax layer. In the second stage, the final portion of the MobileNetV2 backbone was selectively unfrozen for fine-tuning while Batch Normalization layers remained frozen.

The frozen model achieved a validation loss of **0.8218**, with a validation accuracy of approximately **73.22%** at the selected checkpoint. After fine-tuning, validation performance improved substantially to **80.13% accuracy** with a validation loss of **0.6394**. The best fine-tuned checkpoint was obtained at the **third fine-tuning epoch**. On the independent test set, the selected MobileNetV2 model achieved **81.25% accuracy** with a test loss of **0.6308**.

In terms of class-balanced evaluation, MobileNetV2 achieved a **macro precision of 86.06%**, **macro recall of 81.75%**, and **macro-F1 of 81.60%**. Its weighted-F1 score was **81.11%**. The model also showed strong overall class-separation capability, obtaining a **macro ROC-AUC of 0.9873** and a **macro PR-AUC of 0.9018**.

Performance varied across individual malware families. The model classified several classes particularly well, including **Vilsel (F1 = 1.0000)**, **VBA (0.9967)**, **Adposhel (0.9931)**, **Snarasite (0.9900)**, and **Regrun (0.9890)**. However, weaker performance was observed for classes such as **InstallCore (F1 = 0.2428)**, **Sality (0.5250)**, **VBKrypt (0.5792)**, **Neshta (0.5810)**, and **Agent (0.6154)**. In particular, InstallCore showed very low recall (**14.00%**), indicating that many samples from this family were misclassified.

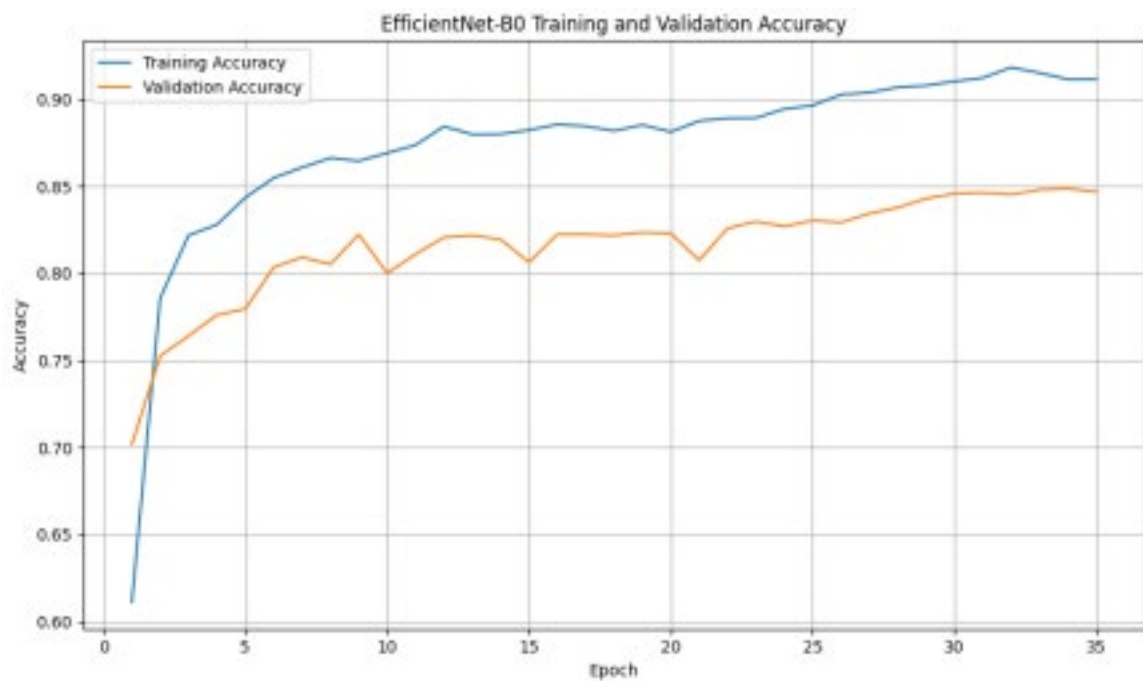
The final MobileNetV2 model contained approximately **2.29 million parameters**, of which about **1.54 million were trainable** during fine-tuning, and the saved model size was approximately **21.07 MB**. Overall, MobileNetV2 provided a relatively compact and effective transfer-learning baseline, achieving good test performance and strong ROC-AUC, although its uneven class-level performance indicates that some malware families remained difficult to distinguish.

5.3 EfficientNet-B0

EfficientNet-B0 is used as another transfer-learning baseline. It provides a strong accuracy–efficiency reference against which the custom CNN and the attention-enhanced model can be compared.

Table:EfficientNet-B0 Block-by-Block Architecture

Stage	Block Type	Expansion Factor	Kernel Size	Stride	Channels (C)	Layers(N)	Output Resolution
1	Conv2D 3 x 3	—	3 x 3	2	32	1	150x150x32
2	MBConv1	1	3 x 3	1	16	1	150x150x16
3	MBConv6	6	3 x 3	2	24	2	75 x75x24
4	MBConv6	6	5 x 5	2	40	2	38x38x40
5	MBConv6	6	3 x 3	2	80	3	19x19x80
6	MBConv6	6	5 x 5	1	112	3	19 x 19 x 112
7	MBConv6	6	5 x 5	2	192	4	10x10x192
8	MBConv6	6	3 x 3	1	320	1	10x10x320
9	Conv2D+GAP +Dense	-	1 x 1	1	1280→26	1	1x1x26



6. Baseline Evaluation

The course brief requires baseline evaluation using the full metric set together with classification reports, training time, parameter count and model size.

```
=====
Final Generalized CNN Results
=====
Validation Accuracy : 74.24%
Validation Loss      : 1.0178
Test Accuracy       : 75.21%
Test Loss           : 0.9747
Weighted Precision  : 79.46%
Weighted Recall     : 75.21%
Weighted F1-score   : 73.87%
=====
```

7. Proposed CNN + CBAM Attention Model

The proposed model extends the custom CNN by adding a Convolutional Block Attention Module (CBAM). The attention module is intended to help the network emphasize informative feature channels and spatial regions.

The selected attention type is CBAM, a combined channel-and-spatial attention mechanism. The course brief permits channel, spatial, SE, CBAM, self-attention, multi-head, or hybrid attention mechanisms and requires the choice and its motivation to be explained.

7.1 Why CBAM?

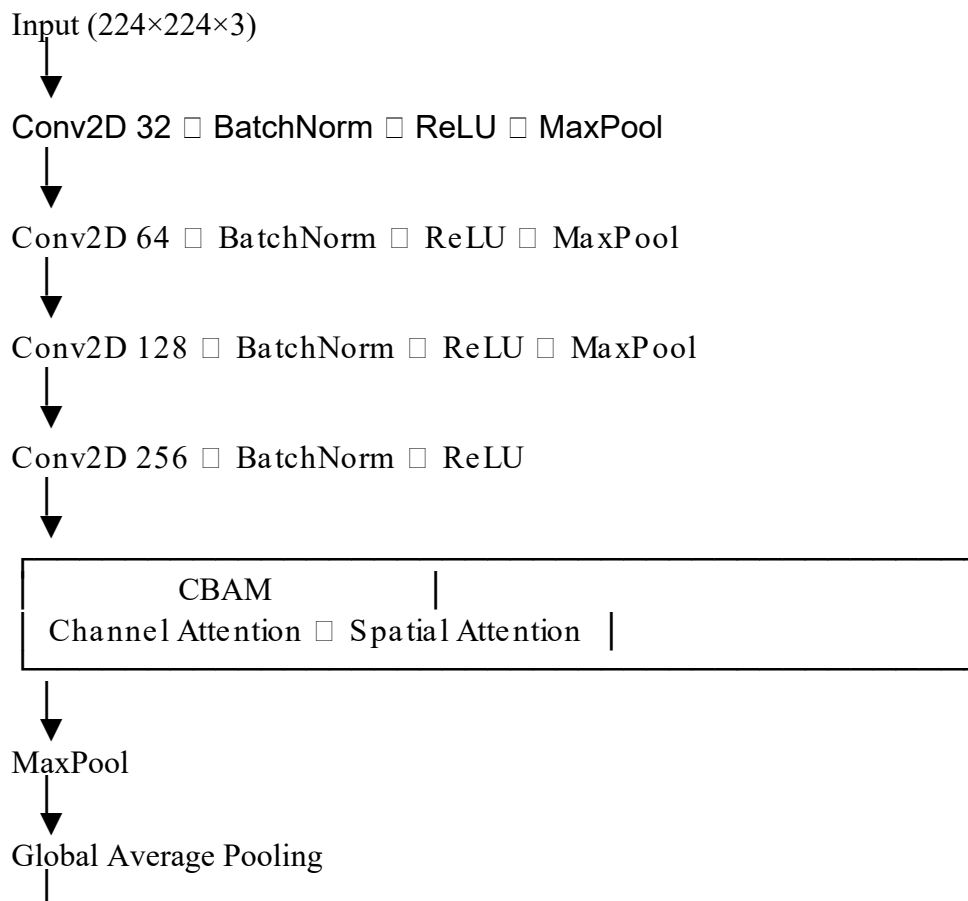
CBAM was selected because it provides two complementary forms of attention. Channel attention learns which feature channels are important, while spatial attention learns which spatial locations are important. For malware images containing distinctive textures, bands, blocks and other byte-derived visual structures, this can help the CNN focus on more informative patterns.

Feature/Parameter	Baseline Architecture	CBAM Model Architecture
Dataset & Preprocessing	Standard Malevis setup	Unchanged from baseline
Input Dimensions	224 x 224 x 3	224 x 224 x 3
CNN Filter Stages	32→64→128→256	32→64→128→256
Convolutional Kernel size	3 x 3	3 x 3
Attention Module	None	CBAM

CBAM Module Count	0	1
Attention Module Placement	N/A	After Conv Block 4 ReLU, before Block 4 MaxPooling
Channel Reduction Ratio	N/A	16
Spatial-Attention Kernel	N/A	7 x 7
Dense Layer Units	128	128
Dropout Rate	0.40	0.40
Optimizer/LR/Loss	Standard Baseline Configuration	Unchanged from baseline

7.2 Proposed Architecture

The proposed architecture follows the custom CNN backbone and inserts CBAM after the final convolutional feature block and before the subsequent pooling/classification stage, as implemented in the project notebook.



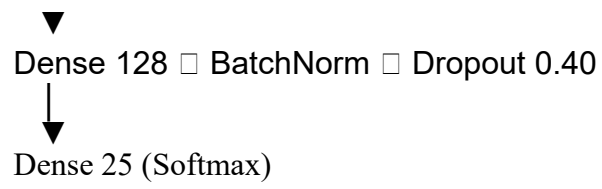
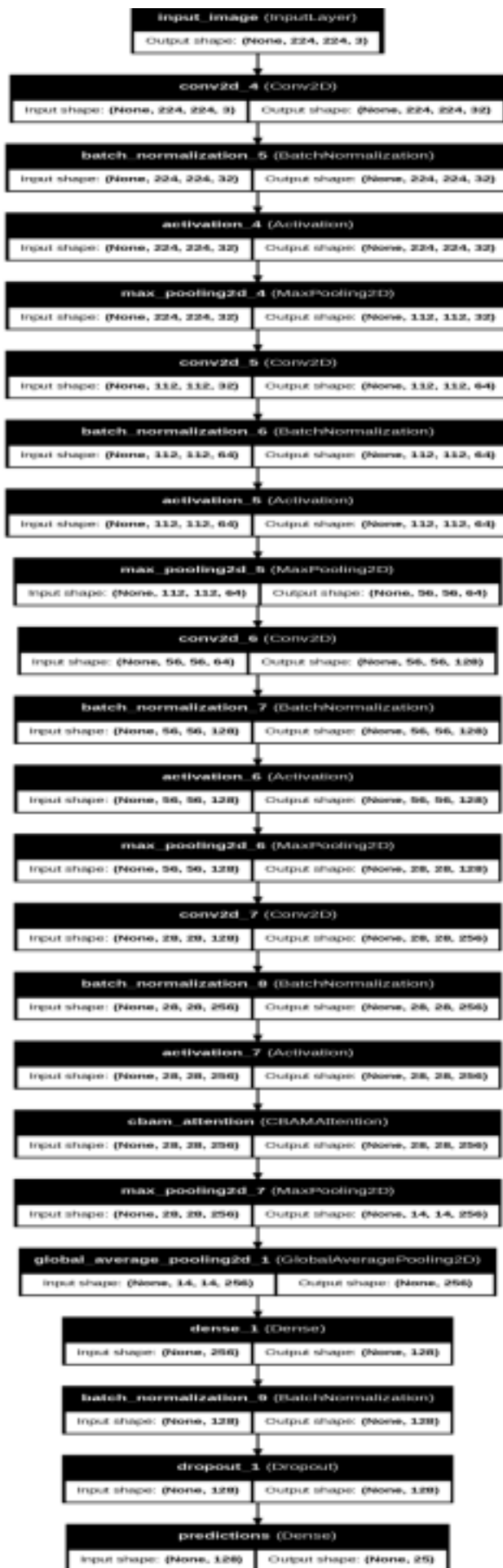


Figure 9. Proposed CNN + CBAM architecture.



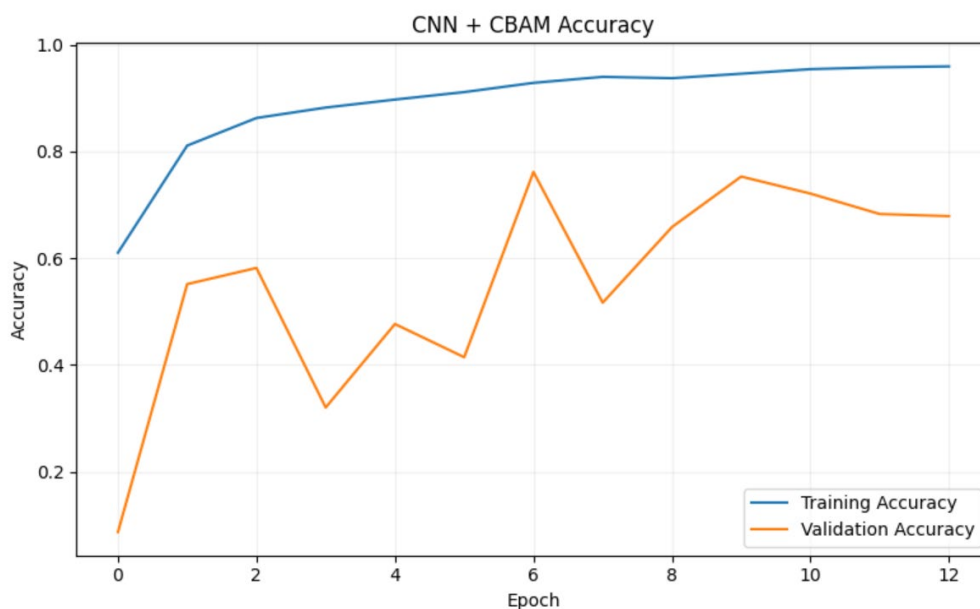
Recommended architecture text to customize with the exact notebook settings:

- Input → Conv(32) → BatchNorm → ReLU → MaxPool
- Conv(64) → BatchNorm → ReLU → MaxPool
- Conv(128) → BatchNorm → ReLU → MaxPool
- Conv(256) → BatchNorm → ReLU → CBAM → MaxPool
- Global Average Pooling → Dropout/Classifier → 26-class output

8. First Results and Comparison

The first proposed-model experiment is compared directly with the custom CNN baseline. The reported first result should answer the central Task 2 question: does adding attention help?

```
=====
CNN + CBAM Test Metrics
=====
Accuracy          : 79.81%
Balanced Accuracy  : 77.40%
Macro Precision    : 81.76%
Macro Recall       : 77.40%
Macro F1-score     : 76.77%
Weighted Precision : 84.87%
Weighted Recall    : 79.81%
Weighted F1-score  : 79.36%
Macro ROC-AUC (OvR) : 0.9819
Cohen's Kappa      : 0.7894
MCC                : 0.7922
=====
```



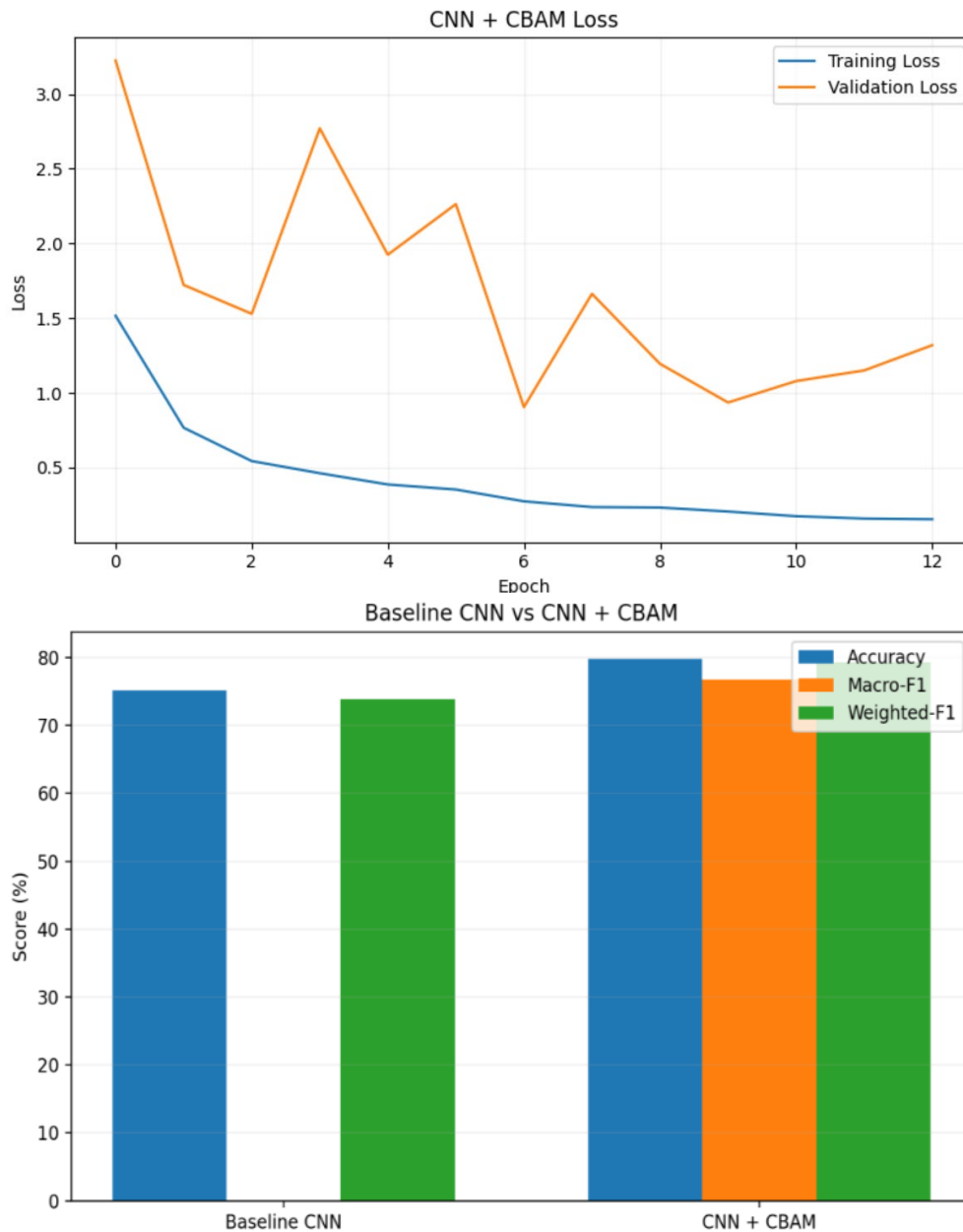


Figure 11. Training history of the proposed CNN + CBAM model.

9. Discussion

The first experiment indicates that the attention-enhanced CNN can improve overall classification accuracy and weighted F1 compared with the custom CNN baseline. The gain suggests that attention may help the network emphasize informative visual patterns. However, the slight reduction in macro F1 indicates that improvements in overall performance do not necessarily translate into equal improvements for every malware family.

The EfficientNet-B0 baseline achieved 85.68% test accuracy in the current experiment, making it an important reference point. Therefore, the proposed CNN + CBAM should not be described as the best model solely because it improves over the custom CNN; its result should be discussed relative to all baselines.

Final comparison table is provided below in the report:

Model	Test Accuracy	Macro F1	Weighted F1	Notes
Generalized CNN	75.21%	74.40%	73.87%	Custom CNN baseline
MobileNetV2	81.25%	81.60%	81.11%	Fine-tuned lightweight transfer-learning baseline
EfficientNet-B0	85.68%	86.18%	85.93%	Best recorded baseline
CNN+CBAM	79.81%	76.77%	79.36%	Attention-Enhanced proposed model

10. Conclusion

Task 2 established CNN-based baselines and developed a custom CNN enhanced with CBAM attention for multi-class malware image classification. The baseline experiments provide reference performance, while the proposed model tests whether channel and spatial attention can improve the custom CNN. The first results show an accuracy improvement over the baseline CNN, while the macro-F1 result indicates that further investigation is needed for class-balanced performance. Task 3 will therefore be used to improve the model, conduct ablation, perform stronger comparisons and add explainability as required by the course project.